

# Module Interface Specification for Software Engineering

Team 11, technically functional

Matthew Huynh

Cieran Diebolt

Vaisnavi Shanthamoorthy

Maham Siddiqui

Eman Ashraf

November 10, 2025

# 1 Revision History

| Date   | Version | Notes |
|--------|---------|-------|
| Date 1 | 1.0     | Notes |
| Date 2 | 1.1     | Notes |

## 2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [<https://github.com/M9Huynh/technically-functional/blob/main/docs/SRS/S>—SS]

[Also add any additional symbols, abbreviations or acronyms —SS]

# Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Revision History</b>                    | <b>i</b>  |
| <b>2</b> | <b>Symbols, Abbreviations and Acronyms</b> | <b>ii</b> |
| <b>3</b> | <b>Introduction</b>                        | <b>1</b>  |
| <b>4</b> | <b>Notation</b>                            | <b>1</b>  |
| <b>5</b> | <b>Module Decomposition</b>                | <b>1</b>  |
| <b>6</b> | <b>Variables</b>                           | <b>2</b>  |
| <b>7</b> | <b>MIS</b>                                 | <b>3</b>  |
| 7.1      | Module - Database Management . . . . .     | 3         |
| 7.1.1    | Uses . . . . .                             | 3         |
| 7.1.2    | Syntax . . . . .                           | 4         |
| 7.1.3    | Semantics . . . . .                        | 4         |
| 7.1.4    | Environment Variables . . . . .            | 5         |
| 7.1.5    | Assumptions . . . . .                      | 5         |
| 7.1.6    | Access Routine Semantics . . . . .         | 5         |
| 7.1.7    | Local Functions . . . . .                  | 6         |
| 7.2      | Module - Generator . . . . .               | 7         |
| 7.2.1    | Uses . . . . .                             | 7         |
| 7.2.2    | Syntax . . . . .                           | 7         |
| 7.2.3    | Semantics . . . . .                        | 7         |
| 7.2.4    | Environment Variables . . . . .            | 8         |
| 7.2.5    | Assumptions . . . . .                      | 8         |
| 7.2.6    | Access Routine Semantics . . . . .         | 8         |
| 7.3      | Module - Home . . . . .                    | 8         |
| 7.3.1    | Uses . . . . .                             | 8         |
| 7.3.2    | Syntax . . . . .                           | 8         |
| 7.3.3    | Semantics . . . . .                        | 8         |
| 7.3.4    | Environment Variables . . . . .            | 8         |
| 7.3.5    | Assumptions . . . . .                      | 9         |
| 7.3.6    | Access Routine Semantics . . . . .         | 9         |
| 7.3.7    | Local Functions . . . . .                  | 9         |
| 7.4      | Module - Main . . . . .                    | 10        |
| 7.4.1    | Uses . . . . .                             | 10        |
| 7.4.2    | Syntax . . . . .                           | 10        |
| 7.4.3    | Semantics . . . . .                        | 10        |
| 7.4.4    | Assumptions . . . . .                      | 10        |
| 7.4.5    | Access Routine Semantics . . . . .         | 10        |

|     |       |                                    |    |
|-----|-------|------------------------------------|----|
|     | 7.4.6 | Local Functions . . . . .          | 11 |
|     | 7.4.7 | Local Functions . . . . .          | 12 |
| 7.5 |       | Module - Analyze . . . . .         | 12 |
|     | 7.5.1 | Uses . . . . .                     | 12 |
|     | 7.5.2 | Syntax . . . . .                   | 12 |
|     | 7.5.3 | Semantics . . . . .                | 13 |
|     | 7.5.4 | Assumptions . . . . .              | 13 |
|     | 7.5.5 | Access Routine Semantics . . . . . | 13 |
|     | 7.5.6 | Local Functions . . . . .          | 13 |
| 7.6 |       | Module - Feedback . . . . .        | 14 |
|     | 7.6.1 | Uses . . . . .                     | 14 |
|     | 7.6.2 | Syntax . . . . .                   | 14 |
|     | 7.6.3 | Semantics . . . . .                | 14 |
|     | 7.6.4 | Assumptions . . . . .              | 14 |
|     | 7.6.5 | Access Routine Semantics . . . . . | 14 |
|     | 7.6.6 | Local Functions . . . . .          | 16 |
|     | 7.6.7 | Local Functions . . . . .          | 17 |
| 7.7 |       | Module - Metrics . . . . .         | 17 |
|     | 7.7.1 | Uses . . . . .                     | 17 |
|     | 7.7.2 | Syntax . . . . .                   | 17 |
|     | 7.7.3 | Semantics . . . . .                | 17 |
|     | 7.7.4 | Assumptions . . . . .              | 17 |
|     | 7.7.5 | Access Routine Semantics . . . . . | 17 |
|     | 7.7.6 | Local Functions . . . . .          | 19 |
|     | 7.7.7 | Local Functions . . . . .          | 19 |
| 7.8 |       | Module - Draw . . . . .            | 19 |
|     | 7.8.1 | Uses . . . . .                     | 19 |
|     | 7.8.2 | Syntax . . . . .                   | 19 |
|     | 7.8.3 | Semantics . . . . .                | 20 |
|     | 7.8.4 | Assumptions . . . . .              | 20 |
|     | 7.8.5 | Access Routine Semantics . . . . . | 20 |
|     | 7.8.6 | Local Functions . . . . .          | 21 |
|     | 7.8.7 | Local Functions . . . . .          | 22 |
| 7.9 |       | Module - UI . . . . .              | 22 |
|     | 7.9.1 | Uses . . . . .                     | 22 |
|     | 7.9.2 | Syntax . . . . .                   | 22 |
|     | 7.9.3 | Semantics . . . . .                | 22 |
|     | 7.9.4 | Assumptions . . . . .              | 22 |
|     | 7.9.5 | Access Routine Semantics . . . . . | 22 |
|     | 7.9.6 | Local Functions . . . . .          | 24 |
|     | 7.9.7 | Local Functions . . . . .          | 25 |

## 8 Appendix

26

## 3 Introduction

The following document details the Module Interface Specifications for a Physiotherapy Companion app. The application is designed to identify users using computer vision for their prescribed physiotherapy exercise. In addition, an analysis will be performed on the user's movement and, after completion, the app will provide metrics to the user on their performance.

This tool utilizes OpenCV and MediaPipe for image recognition and then performs measurements on the number of repetitions, range of motion and perceived exertion.

Complementary documents include the [System Requirement Specifications](#) and [Module Guide](#). The full documentation and implementation can be found at <https://github.com/M9Huynh/technically-functional>.

## 4 Notation

[You should describe your notation. You can use what is below as a starting point. —SS]

The structure of the MIS for modules comes from [Hoffman and Strooper \(1995\)](#), with the addition that template modules have been adapted from [Ghezzi et al. \(2003\)](#). The mathematical notation comes from Chapter 3 of [Hoffman and Strooper \(1995\)](#). For instance, the symbol  $:=$  is used for a multiple assignment statement and conditional rules follow the form  $(c_1 \Rightarrow r_1 | c_2 \Rightarrow r_2 | \dots | c_n \Rightarrow r_n)$ .

The following table summarizes the primitive data types used by Software Engineering.

| Data Type      | Notation     | Description                                                       |
|----------------|--------------|-------------------------------------------------------------------|
| character      | char         | a single symbol or digit                                          |
| integer        | $\mathbb{Z}$ | a number without a fractional component<br>in $(-\infty, \infty)$ |
| natural number | $\mathbb{N}$ | a number without a fractional component<br>in $[1, \infty)$       |
| real           | $\mathbb{R}$ | any number in $(-\infty, \infty)$                                 |

The specification of Software Engineering uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, Software Engineering uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

## 5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

| Level 1           | Level 2                                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Hardware-Hiding   |                                                                                                                                                       |
| Behaviour-Hiding  | Input Parameters<br>Output Format<br>Output Verification<br>Temperature ODEs<br>Energy Equations<br>Control Module<br>Specification Parameters Module |
| Software Decision | Sequence Data Structure<br>ODE Solver<br>Plotting                                                                                                     |

Table 1: Module Hierarchy

## 6 Variables

Table 2: Variable used throughout MIS

| Name            | Type          | Description                                                             | Used in sections |
|-----------------|---------------|-------------------------------------------------------------------------|------------------|
| Name            | String        | Name of the user                                                        | 7.1              |
| Role            | String        | Values can be PT or patient                                             | 7.1              |
| email           | String        | Used as login ID                                                        | 7.1              |
| password        | String        | Set by user for login purposes                                          | 7.1              |
| birthday        | String        | User's birthday for verification                                        | 7.1              |
| license_no      | Int           | Verification of PT                                                      | 7.1              |
| invite_code     | String        | Given by PT to patient for registration                                 | 7.1              |
| Date            | String        | Date selected by user                                                   | 7.1              |
| Time            | String        | Time selected by user                                                   | 7.2              |
| Recent_analysis | ADT           | Most recently generated analysis                                        | 7.1              |
| Recent_report   | Record        | Generated report of recent_video                                        | TBD              |
| Recent_video    | String (JSON) | Last video recorded by patient                                          | TBD              |
| Video           | String (JSON) | A video in the user database                                            | TBD              |
| Report          | Record        | Report(s) sent to Generator module for analysis OR requested by patient | TBD              |

Table 2: Variable used throughout MIS (continued)

| Name            | Type   | Description                                    | Used in sections |
|-----------------|--------|------------------------------------------------|------------------|
| Ex_instruct     | String | Performance instructions of requested exercise | TBD              |
| Sets            | String | Number of sets given by PT                     | 7.1              |
| Repetitions     | String | Repetitions of exercise per set                | 7.1              |
| Rest_time       | String | Rest time between sets                         | 7.1              |
| Leg_landmarks   | Array  | MediaPipe output of leg                        | 7.4              |
| Processed_frame | Image  | Video frame with overlaid markers              | 7.4              |

## 7 MIS

### 7.1 Module - Database Management

*ADT*

This module creates an instance of UserAccount\_P/PT and is an adapter that connects to the user database. This database consists of these tables:

1. User data: this consists of information such as name, role, email, password, birthday, and license\_no/invite\_code.
2. Exercise data: this table consists of exercise nstructions, maximum height (the highest point their leg can lift up to), minimum height (starting point) and UserAccount\_P['Name'].
3. User activity: This will contain information about a specific patient's exercise activity. Data such as exercise duration, maximum height, number of repetitions, target area, the date the exercise was performed and its time, rest time, number of sets and analysis. Additionally, it will include a column for the patient's feedback on their exercise performance.

#### 7.1.1 Uses

- name
- role
- email
- password



- birthday
- license\_no
- invite\_code
- exercise
- time
- date
- duration
- Recent\_video
- Recent\_analysis
- Sets
- repetitions
- rest\_time

### 7.1.2 Syntax

#### Exported Constants

- *UserAccount\_P*: ADT
- *UserAccount\_PT*: ADT

#### Exported Access Programs

### 7.1.3 Semantics

#### State Variables

*A*: an instance of a patient UserAccount

*B*: an instance of a patient UserAccount

*C*: an instance of a PT UserAccount

#### State Invariant

$\forall A, B, C \in UserDatabase : A[i] \neq B[j] \neq C[k]$   
 $0 < A < length(A) \cap A[i] \notin \emptyset$

| Access Routine Name          | Input                                             | Output                                | Exceptions                                               |
|------------------------------|---------------------------------------------------|---------------------------------------|----------------------------------------------------------|
| get_userdb_info              | NA                                                | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error                         |
| PTaccount_delete             | Name<br>email                                     | NA                                    | required_field_empty<br>patient_not_found                |
| username_pw_match            | Email<br>Password                                 | NA                                    | wrong_email_<br>password                                 |
| add_analysis.to_<br>user_act | UserAccount_P<br>Recent_analysis                  | NA                                    | User_not_found<br>Upload_error                           |
| save_video                   | UserAccount_P<br>Recent_video                     | NA                                    | User_not_found<br>Upload_error<br>Video_too_short        |
| get_analysis                 | UserAccount_P<br>Date                             | Recent_analysis                       | User_not_found<br>Download_error<br>Analysis_not_found   |
| exerc_instruct               | UserAccount_P<br>Exercise                         | Ex_instruct                           | User_not_found<br>Download_error<br>exercise_not_in_list |
| PT_modifies_exercise         | UserAccount_P<br>Sets<br>Repetitions<br>Rest_time | Ex_instruct                           | User_not_found<br>Download_error<br>exercise_not_in_list |

Table 3: Exported Access Routines

#### 7.1.4 Environment Variables

*Screen*: The application's display to the user

*Touchscreen*: Users will click on the screen to select their desired functionality

#### 7.1.5 Assumptions

NA.

#### 7.1.6 Access Routine Semantics

**get\_userdb\_info()**:

NA (*There is no transition taking place*)

### PTaccount\_delete():

1. From the home screen, PT accesses their patient list
2. PT selects the desired patient's name
3. PT is directed to profile of patient
4. PT selects '*Delete Patient*'
5. The selected patient is deleted from the user database

#### 7.1.7 Local Functions

| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| account_create      | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |
| set_user_info       | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | NA                                    | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |
| send_db_analysis    | <i>UserAccount_P</i>                                                                                                         | Recent_analysis                       | User_not_found<br>Download_error                                                                                 |
| send_db_video       | <i>UserAccount_P</i>                                                                                                         | Recent_video                          | User_not_found<br>Download_error                                                                                 |
| set_analysis_report | <i>UserAccount_P</i>                                                                                                         | Recent_analysis                       | User_not_found<br>Download_error                                                                                 |

Table 4: Account Management Access Routines

## 7.2 Module - Generator

This module's purpose is to display the most recent report, an overall analysis of progress, and rehabilitation insights.

### 7.2.1 Uses

- Recent\_report
- Report\_db
- UserAccount\_P

### 7.2.2 Syntax

**Exported Constants**

**Exported Access Programs**

| Access Routine Name | Input                      | Output               | Exceptions                                              |
|---------------------|----------------------------|----------------------|---------------------------------------------------------|
| generate_insights   | NA                         | NA                   | <i>No_activity_history</i><br><i>Cannot_connect_API</i> |
| generate_report     | <i>Date</i><br><i>Time</i> | <i>Recent_report</i> | NA                                                      |
| get_report          | <i>Date</i><br><i>Time</i> | <i>Report</i>        | NA                                                      |

Table 5: Generator Access Routines

### 7.2.3 Semantics

**State Variables**

*A*: an instance of UserAccount\_P

*E*: an instance of Recent\_analysis

*V*: video

*R*: report

*RR*: recent\_report

**State Invariant**

$\forall RR, \exists(A \cap V)$

$\forall R, \exists(A \cap V_i | i \geq 1..n, n \in 1..N)$

#### 7.2.4 Environment Variables

NA

#### 7.2.5 Assumptions

#### 7.2.6 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

### 7.3 Module - Home

This module displays past exercises, allows viewing of user profile, and directs to the Generator module. PTs are able to comment on their patient's submissions.

#### 7.3.1 Uses

#### 7.3.2 Syntax

##### Exported Constants

NA

##### Exported Access Programs

#### 7.3.3 Semantics

##### State Variables

*A*: an instance of UserAccount

*B*: an instance of UserAccount

##### State Invariant

NA

#### 7.3.4 Environment Variables

NA

| Access Routine Name | Input                           | Output                                | Exceptions                                |
|---------------------|---------------------------------|---------------------------------------|-------------------------------------------|
| view_profile        | <i>Name</i>                     | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error          |
| display_report      | <i>Name</i><br><i>email</i>     | NA                                    | required_field_empty<br>patient_not_found |
| display_summary     | <i>Email</i><br><i>Password</i> | NA                                    | wrong_email_<br>password                  |
| p_past_exercises    | NA                              | NA                                    | no_activity                               |
| PT_feedback_on_P    | NA                              | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error          |

Table 6: Home Exported Access Routines

### 7.3.5 Assumptions

### 7.3.6 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.3.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

| Access Routine Name | Input | Output                                | Exceptions       |
|---------------------|-------|---------------------------------------|------------------|
| get_profile_gen     | NA    | UserAccount_P<br>OR<br>UserAccount_PT | Cannot_access_db |

Table 7: Account Management Access Routines

## 7.4 Module - Main

This module handles the video recording, placing markers on it and checking if the patient's form is appropriate for performing the exercise through the use of external libraries (notably OpenCV).

### 7.4.1 Uses

### 7.4.2 Syntax

**Exported Constants**

**Exported Access Programs**

### 7.4.3 Semantics

**State Variables**

**State Invariant**

### 7.4.4 Assumptions

### 7.4.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

| Access Routine Name | Input | Output                                | Exceptions                            |
|---------------------|-------|---------------------------------------|---------------------------------------|
| record_video        | NA    | Recent_video                          | Insufficient_data<br>Unable_to_record |
| retry_record        | NA    | Recent_video                          | Unable_to_record                      |
| draw_markers        | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error      |
| form_vs_template    | NA    | NA                                    | User_not_found<br>Download_error      |
| submit_video        | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error      |
| get_metrics         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error      |
| get_rt_feed         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error      |

Table 8: Main Exported Access Routines

#### 7.4.6 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]



| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| placeholder         | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |

Table 9: Account Management Access Routines

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

#### 7.4.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

## 7.5 Module - Analyze

NA

### 7.5.1 Uses

### 7.5.2 Syntax

**Exported Constants**

**Exported Access Programs**

| Access Routine Name | Input | Output                                | Exceptions                       |
|---------------------|-------|---------------------------------------|----------------------------------|
| get_metrics         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

| Access Name               | Routine | Input | Output                                | Exceptions                       |
|---------------------------|---------|-------|---------------------------------------|----------------------------------|
| send_analysis_to_feedback |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| analysis_rt               |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| analysis_overall          |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| adjust_metrics            |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| update_metrics            |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| send_analysis_to_db       |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

### 7.5.3 Semantics

#### State Variables

#### State Invariant

### 7.5.4 Assumptions

#### 7.5.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

### 7.5.6 Local Functions

| Access Name    | Routine | Input | Output                                | Exceptions                       |
|----------------|---------|-------|---------------------------------------|----------------------------------|
| adjust_metrics |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

## 7.6 Module - Feedback

NA

### 7.6.1 Uses

### 7.6.2 Syntax

Exported Constants

Exported Access Programs

| Access Name | Routine | Input | Output                                | Exceptions                       |
|-------------|---------|-------|---------------------------------------|----------------------------------|
| placeholder |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

### 7.6.3 Semantics

State Variables

State Invariant

### 7.6.4 Assumptions

### 7.6.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.6.6 Local Functions

| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| placeholder         | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |

Table 13: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.6.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

## 7.7 Module - Metrics

NA

### 7.7.1 Uses

### 7.7.2 Syntax

Exported Constants

Exported Access Programs

| Access Name | Routine | Input | Output                                | Exceptions                       |
|-------------|---------|-------|---------------------------------------|----------------------------------|
| placeholder |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

### 7.7.3 Semantics

State Variables

State Invariant

### 7.7.4 Assumptions

### 7.7.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.7.6 Local Functions

| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| placeholder         | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |

Table 15: Account Management Access Routines

### 7.7.7 Local Functions

## 7.8 Module - Draw

NA

### 7.8.1 Uses

### 7.8.2 Syntax

Exported Constants

Exported Access Programs

| Access Routine Name | Input | Output                                | Exceptions                       |
|---------------------|-------|---------------------------------------|----------------------------------|
| record_check        | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| form_check          | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |
| draw_points         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |



### 7.8.3 Semantics

#### State Variables

#### State Invariant

### 7.8.4 Assumptions

### 7.8.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]
- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.8.6 Local Functions

| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| mpipe_draw          | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |

Table 17: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.8.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

## 7.9 Module - UI

NA

### 7.9.1 Uses

### 7.9.2 Syntax

Exported Constants

Exported Access Programs

| Access Name | Routine | Input | Output                                | Exceptions                       |
|-------------|---------|-------|---------------------------------------|----------------------------------|
| placeholder |         | NA    | UserAccount_P<br>OR<br>UserAccount_PT | User_not_found<br>Download_error |

### 7.9.3 Semantics

State Variables

State Invariant

### 7.9.4 Assumptions

### 7.9.5 Access Routine Semantics

[accessProg —SS]():

- transition: [if appropriate —SS]
- output: [if appropriate —SS]

- exception: [if appropriate —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.9.6 Local Functions

| Access Routine Name | Input                                                                                                                        | Output                                | Exceptions                                                                                                       |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------------------|
| placeholder         | <i>Name</i><br><i>Role</i><br><i>Birthday</i><br><i>License_no/</i><br><i>Invite_code</i><br><i>Email</i><br><i>Password</i> | UserAccount_P<br>or<br>UserAccount_PT | Input_format_invalid<br>Required_field_empty<br>Account_already_exists<br>Incorrect_creds<br>Invite_code_expired |

Table 19: Account Management Access Routines

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

[A module without environment variables or state variables is unlikely to have a state transition. In this case a state transition can only occur if the module is changing the state of another module. —SS]

[Modules rarely have both a transition and an output. In most cases you will have one or the other. —SS]

### 7.9.7 Local Functions

[As appropriate —SS] [These functions are for the purpose of specification. They are not necessarily something that is going to be implemented explicitly. Even if they are implemented, they are not exported; they only have local scope. —SS]

## References

Carlo Ghezzi, Mehdi Jazayeri, and Dino Mandrioli. *Fundamentals of Software Engineering*. Prentice Hall, Upper Saddle River, NJ, USA, 2nd edition, 2003.

Daniel M. Hoffman and Paul A. Strooper. *Software Design, Automated Testing, and Maintenance: A Practical Approach*. International Thomson Computer Press, New York, NY, USA, 1995. URL <http://citeseer.ist.psu.edu/428727.html>.

## 8 Appendix

[Extra information if required —SS]

## Appendix — Reflection

[Not required for CAS 741 projects —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Problem Analysis and Design.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing “what you think the evaluator wants to hear.”

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?
2. What pain points did you experience during this deliverable, and how did you resolve them?
3. Which of your design decisions stemmed from speaking to your client(s) or a proxy (e.g. your peers, stakeholders, potential users)? For those that were not, why, and where did they come from?
4. While creating the design doc, what parts of your other documents (e.g. requirements, hazard analysis, etc), if any, needed to be changed, and why?
5. What are the limitations of your solution? Put another way, given unlimited resources, what could you do to make the project better? (LO\_ProbSolutions)
6. Give a brief overview of other design solutions you considered. What are the benefits and tradeoffs of those other designs compared with the chosen design? From all the potential options, why did you select the documented design? (LO\_Explores)